Solend Steamm

Security Assessment

February 4th, 2025 — Prepared by OtterSec

Sangsoo Kang	sangsoo@osec.io
Michał Bochnak	embe221ed@osec.io
Robert Chen	r@osec.io

Table of Contents

01 — Executive Summary

Overview

Solend engaged OtterSec to assess the `steamm` program. This assessment was conducted between January 18th and January 31st, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 12 findings throughout this audit engagement.

In particular, we identified a vulnerability in which inadequate minimum liquidity may result in inflation attacks in the bank module (OS-SAM-ADV-00). Additionally, the assertion check for the CToken ratio may fail due to the presence of roundings during cToken-to-underlying token conversions in the recall functionality, resulting in frequent aborts (OS-SAM-ADV-01). Furthermore, the current tokens-to-deposit logic allows users to over-supply tokens due to insufficient checks, which may disrupt the pool's balance (OS-SAM-ADV-02).

We also made suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices (OS-SAM-SUG-04), and advised introducing a minimum liquidity reserve to prevent the pool from becoming entirely empty (OS-SAM-SUG-00). We further recommended implementing validation logic in several areas within the codebase to mitigate potential security issues (OS-SAM-SUG-01).

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/solendprotocol/steamm. This audit was performed against commit ecfdf45. We further reviewed PR#81.

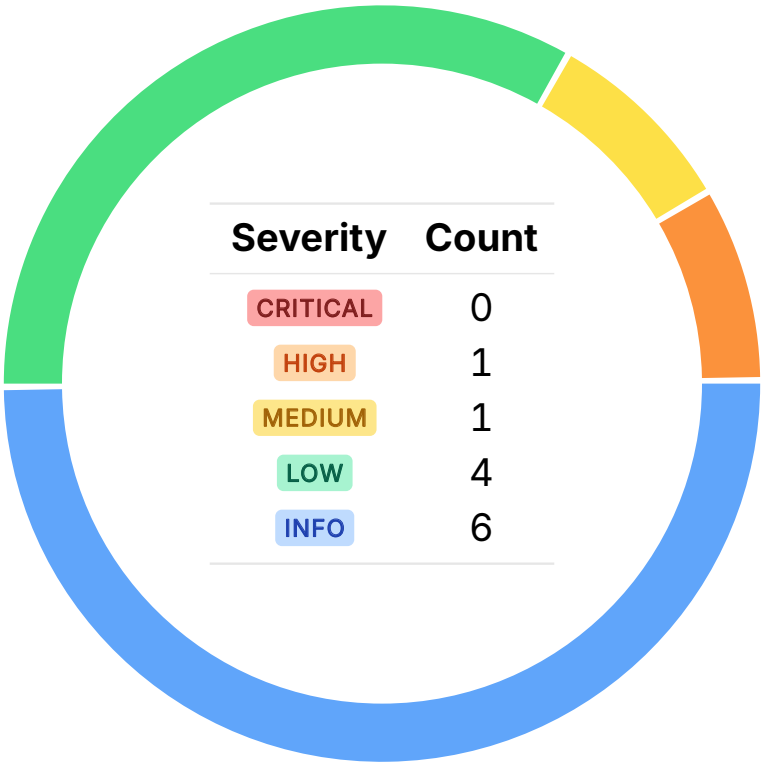
A brief description of the program is as follows:

Name	Description
steamm	Steamm is an AMM with embedded money market integration, designed to maximize capital efficiency. It improves liquidity utilization by depositing idle funds into Suilend's lending markets, generating extra yield for liquidity providers.

03 — Findings

Overall, we reported 12 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	1
MEDIUM	1
LOW	4
INFO	6

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-SAM-ADV-00	HIGH	RESOLVED	Inadequate minimum liquidity may result in inflation attacks in `bank`.
OS-SAM-ADV-01	MEDIUM	RESOLVED	The assertion check for `CTokenRatio` may fail due to the presence of roundings during `cToken`-to-underlying token conversions in `recall`, resulting in frequent aborts.
OS-SAM-ADV-02	LOW	RESOLVED	The current `tokens_to_deposit` logic allows users to oversupply tokens due to insufficient checks, which may disrupt the pool's balance.
OS-SAM-ADV-03	LOW	RESOLVED	`amount_to_recall` in `recall` may exceed the available reserves due to the utilization of `max(min_token_block_size)`, resulting in insufficient liquidity for the recall process.
OS-SAM-ADV-04	LOW	RESOLVED	`bank` fails to fully manage interactions with the `LendingMarket`, allowing external modifications to the obligation's state and potential changes to the `ctokens` balance without the involvement of `bank`.
OS-SAM-ADV-05	LOW	RESOLVED	If one reserve is fully drained, functions such as `tokens_to_deposit` and `lp_tokens_to_mint` will fail due to a division by zero error.

Lack of Minimum Liquidity Constraint HIGH OS-SAM-ADV-00

Description

The `bank` module does not have any constrains on the minimum amount of liquidity that should be present at all times. Insufficient minimum liquidity may expose it to inflation attacks, enabling malicious actors to manipulate the value of `bToken`. For example, If the value of bToken exceeds a 1:1 ratio, burning bToken and increasing the amount of the underlying token can trigger zero mint when a user deposits, causing a loss for the user.

Remediation

Enforce a minimum liquidity requirement in `bank` to prevent inflation attacks.

Patch

Resolved in 6e17d4f.

Assertion Failure Due to Rounding MEDIUM OS-SAM-ADV-01

Description

`bank::recall` compares the two products in an assert statement to verify the following equation.

$$\frac{bank.funds\_deployed}{lending.ctokens} <= \frac{recalled\_amount}{ctoken\_amount}$$

However, due to the presence of roundings during the conversion between `cTokens` and underlying tokens, the product of `ctoken_amount` and the deployed funds often exceeds the product of the bank's total `CTokens` and recalled amount, failing the assertion check and resulting in frequent aborts.

>_ steamm/sources/bank/bank.move RUST

```
fun recall<P, T, BToken>(
    bank: &mut Bank<P, T, BToken>,
    lending_market: &mut LendingMarket<P>,
    amount_to_recall: u64,
    clock: &Clock,
    ctx: &mut TxContext,
) {
    [...]
     assert!(
        ctoken_amount * bank.funds_deployed(lending_market, clock).floor() <= lending.ctokens *
            ↪  recalled_amount,
        EInvalidCTokenRatio,
    );
    [...]
}
```

Remediation

Remove the assertion since the inequality isn't always guaranteed to hold.

Patch

Resolved in 38c5317.

Overestimation of Tokens Resulting in Oversupply LOW OS-SAM-ADV-02

Description

`pool_math::tokens_to_deposit` attempts to calculate the optimal deposit amounts `a_star` (for token A) and `b_star` (for token B) based on the current reserve ratios while adhering to user-specified maximum constraints `max_a` and `max_b`. However, the approach may overestimate the required tokens under certain conditions.

>_ steamm/sources/pool/pool_math.move RUST

```
fun tokens_to_deposit(reserve_a: u64, reserve_b: u64, max_a: u64, max_b: u64): (u64, u64) {
    assert!(max_a > 0, EDepositMaxAParamCantBeZero);
    if (reserve_a == 0 && reserve_b == 0) {
        (max_a, max_b)
    } else {
        let b_star = safe_mul_div_up(max_a, reserve_b, reserve_a);
        if (b_star <= max_b) { (max_a, b_star) } else {
            let a_star = safe_mul_div_up(max_b, reserve_a, reserve_b);
            assert!(a_star > 0, EDepositRatioLeadsToZeroA);
            assert!(a_star <= max_a, EDepositRatioInvalid);
            (a_star, max_b)
        }
    }
}
```

When calculating `b_star`, if `max_a * reserve_b` is not divisible by `reserve_a`, the returned values may include a dust amount. This implies that users may inadvertently attempt to supply more tokens than necessary to maintain the correct reserve balance.

Remediation

Calculate the liquidity based on both `max_a` and `max_b`, considering the pool's current reserve ratio, and then back-calculate the corresponding `a_star` and `b_star`.

Patch

Acknlowedged by the Solend development team.

Risk of Excess Recall Amount LOW OS-SAM-ADV-03

Description

The logic in `bank::recall` may result in a situation where the `amount_to_recall` becomes greater than the available reserves due to the utilization of `max(bank.min_token_block_size)`. The `amount_to_recall` is adjusted to ensure it is at least the size of `bank.min_token_block_size`. On recalling an amount smaller than the `min_token_block_size`, the function will automatically increase the recall amount to the minimum block size. The issue occurs if the available funds (the reserves in the bank) are not large enough to accommodate the adjusted `amount_to_recall`.

Remediation

Ensure that the `amount_to_recall` never exceeds the actual available funds.

Patch

Acknowledged by the Solend development team.

Lack of Synchronization Between Bank and LendingMarket LOW OS-SAM-ADV-04

Description

The vulnerability arises from the lack of full integration between `bank` and `LendingMarket` features. It is possible for the `LendingMarketOwnerCap` to create additional `ObligationOwnerCap` instances. This implies that a new obligation may be created without requiring interaction with the Bank module, which should ideally have control over such state changes. Furthermore, `claim_rewards_and_deposit` allows for the claim of rewards and automatic deposit of `CTokens`, which increases the obligation's `CTokens` balance. As this feature operates outside the control of `bank`, it will not account for these changes in its tracking system.

Remediation

Ensure that the bank and lending market are operated by a trustworthy entity.

Patch

Acknowledged by the developers.

Division by Zero Error LOW OS-SAM-ADV-05

Description

In the current implementation, it is possible for the pool to have one of the reserves fully drained. In `pool_math::tokens_to_deposit`, the core logic calculates the amount of token B (`b_star`) a user should deposit based on the current ratio between the reserves of token A and token B. The function utilizes the `safe_mul_div_up` helper function to scale the deposit amounts accordingly. If `reserve_a == 0`, the division in `safe_mul_div_up` will attempt to divide by zero, resulting in an abort.

>_ steamm/sources/pool/pool_math.move RUST

```
fun tokens_to_deposit(reserve_a: u64, reserve_b: u64, max_a: u64, max_b: u64): (u64, u64) {
    assert!(max_a > 0, EDepositMaxAParamCantBeZero);
    if (reserve_a == 0 && reserve_b == 0) {
        (max_a, max_b)
    } else {
        let b_star = safe_mul_div_up(max_a, reserve_b, reserve_a);
        if (b_star <= max_b) { (max_a, b_star) } else {
            let a_star = safe_mul_div_up(max_b, reserve_a, reserve_b);
            assert!(a_star > 0, EDepositRatioLeadsToZeroA);
            assert!(a_star <= max_a, EDepositRatioInvalid);
            (a_star, max_b)
        }
    }
}
```

Similarly, `lp_tokens_to_mint` calculates the number of LP tokens to mint based on the user's deposit relative to the pool's existing reserves. If one of the reserves (`reserve_a` and `reserve_b` is zero, it will result in division by zero in `safe_mul_div`.

Remediation

Add checks to abort the functions early if either `reserve_a` or `reserve_b` is zero.

Patch

Acknowledged by the developers.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-SAM-SUG-00	The `EmptyBank` check in `compute_utilisation_bps` requires at least one unit of funds in the pool. Introducing a `MINIMUM_LIQUIDITY` constraint ensures a non-zero reserve, allowing for full withdrawals.
OS-SAM-SUG-01	There are several instances where proper validation is not performed, resulting in potential security issues.
OS-SAM-SUG-02	Additional safety checks may be incorporated within the codebase to make it more robust and secure.
OS-SAM-SUG-03	Recommendation for modifying the codebase for improved functionality, efficiency, and mitigate potential security issues.
OS-SAM-SUG-04	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.
OS-SAM-SUG-05	Highlighting inconsistencies that affecting the overall functionality and clarity of the codebase.

Minimum Pool Liquidity OS-SAM-SUG-00

Description

The `EmptyBank` assertion in `bank_math::compute_utilisation_bps` ensures that the sum of `funds_available` and `funds_deployed` is always greater than zero. Without this check, the division in the calculation of the utilization ratio formula will result in a division-by-zero error. However, this creates a problem during full withdrawals.

>_ steamm/sources/bank/bank_math.move RUST

```
public(package) fun compute_utilisation_bps(funds_available: u64, funds_deployed: u64): u64 {
    assert!(funds_available + funds_deployed > 0, EEmptyBank);
    (funds_deployed * 10_000) / (funds_available + funds_deployed)
}
```

If all funds are withdrawn, `funds_available` becomes zero, and if no funds are deployed, then the sum of `funds_available` and `funds_deployed` will be zero. Thus, the assertion will fail, halting execution and preventing the last withdrawal.

Remediation

Introduce a minimum liquidity reserve (such as `MINIMUM_LIQUIDITY` in `pool`) to prevent the pool from becoming entirely empty.

Patch

Resolved in 6e17d4f.

Missing Validation Logic

OS-SAM-SUG-01

Description

1. Include a check to ensure that the two tokens involved in the creation of the pool are of different types.
2. `pool::redeem_liquidity` includes a check for the consistency of the reserve-to-LP supply ratio via `assert_lp_supply_reserve_ratio`, but it only validates the ratio for `coin_a` and ignores doing the same for `coin_b`. Thus, any inconsistency in coin B's reserves will go undetected, resulting in unbalanced pool reserves. Call `assert_lp_supply_reserve_ratio` for `coin_b` as well.
3. `redeem_liquidity`, `deposit_liquidity`, `mint_btokens`, and `burn_btokens` do not validate whether `coin_in` and `coin_out` are zero, which impacts the correctness of these operations. Allowing zero-value minting, burning, deposit, and withdrawing is unnecessary, wastes computational resources, and adds noise to event logs. Ensure `coin_in` and `coin_out` are not zero. Similarly, `pool::swap` should check that the `amount_in` is greater than zero.
4. `pool_math::lp_tokens_to_mint` may return zero in certain edge cases if `amount_a` or `amount_b` is too small relative to `reserve_a` or `reserve_b`. Add appropriate handling for such cases.
5. `FixedPoint64` may be greater than `U64_MAX` particularly after casting results from `to_u128_up` or `mul`. For example in `quoter_math::swap`, during the calculation of `delta_out`, casting directly to `u64` may result in overflow if the intermediate value exceeds `u64::MAX`. Ensure to check the values before casting to `u64`.

>_ steamm/sources/quoters/quoter_math.move RUST

```
public(package) fun swap([...]): u64 {
    [...]
    // `z` is defined as Δout / ReserveOut. Therefore depending on the
    // direction of the trade we pick the corresponding ouput reserve
    let delta_out = if (x2y) {
        z.mul(r_y).to_u128_down() as u64
    } else {
        z.mul(r_x).to_u128_down() as u64
    };
    [...]
}
```

6. Ensure that when a pool is created, the token type is correctly linked to the corresponding oracle index to prevent mismatches. Additionally, prevent modification of the oracle reference to another feed post-deployment, as this may compromise price accuracy and protocol security.

Remediation

Incorporate the above-mentioned validations into the codebase.

Patch

1. Issue #1 resolved in 344e81b.
2. Issue #2 resolved in ab76c55.
3. Issue #3 resolved in a8c5738.

Additional Safety Checks OS-SAM-SUG-02

Description

1. Add a validation step in `pool::deposit_liquidity` to check whether the minted LP tokens (`lp_coins`) are at least equal to `MINIMUM_LIQUIDITY` if this is the first deposit into the pool. This ensures that when the first deposit is made, enough LP tokens exist for the minimum lock.

>_ steamm/sources/pool/pool.move RUST

```
public fun deposit_liquidity<A, B, Quoter: store, LpType: drop>([...]): (Coin<LpType>,
    ↪  DepositResult) {
    pool.version.assert_version_and_upgrade(CURRENT_VERSION);
    assert!(!(coin_a.value() == 0 && coin_b.value() == 0), EEmptyCoins);
    // Compute token deposits and delta lp tokens
    let quote = quote_deposit_(
        pool,
        max_a,
        max_b,
    );
    let initial_lp_supply = pool.lp_supply.supply_value();
    let (initial_total_funds_a, initial_total_funds_b) = pool.balance_amounts();
    let balance_a = coin_a.balance_mut().split(quote.deposit_a());
    let balance_b = coin_b.balance_mut().split(quote.deposit_b());
}
```

2. The `fees` module does not validate input parameters, which may lead to unexpected behavior or errors if incorrect values are provided. Implementing proper validation is recommended to ensure robustness.
3. Validate the subtractions in both `bank_math::compute_amount_to_deploy` and `bank_math::compute_amount_to_recall` to mitigate the possibility of unintended errors during the subtraction operations, especially if the `target_utilisation` changes.
4. Handle the case when `bank.btoken_supply.supply_value()` is less than `MINIMUM_LIQUIDITY` to appropriately address the scenario when the bank's liquidity falls below a minimum threshold required to maintain stable operations.
5. Ensure that `get_price` explicitly asserts the fetched price is greater than zero to prevent the utilization of invalid or zero prices within the protocol.

Remediation

Add the checks stated above.

Patch

1. Issue #1 resolved in 0a27ee9.
2. Issue #2 resolved in 0aa893f.

Code Refactoring OS-SAM-SUG-03

Description

1. Enforce a rebalance after `bank::set_utilisation_bps` is called to ensure that the lending system remains within its desired liquidity utilization range.
2. The early return in `bank::deploy` prevents fund deployment if the amount is smaller than the `min_token_block_size`. However, this may result in improper adjustment of the bank's utilization rate, preventing it from staying within the target utilization range.

>_ steamm/sources/bank/bank.move RUST

```
fun deploy<P, T, BToken>(
    bank: &mut Bank<P, T, BToken>,
    lending_market: &mut LendingMarket<P>,
    amount_to_deploy: u64,
    clock: &Clock,
    ctx: &mut TxContext,
) {
    let lending = bank.lending.borrow();
    if (amount_to_deploy < bank.min_token_block_size) {
        return
    };
    let balance_to_lend = bank.funds_available.split(amount_to_deploy);
    [...]
}
```

3. Swap the order of multiplication and division in `bank::funds_deployed` to prevent large intermediate values, which may result in precision loss. Dividing first and then multiplying ensures higher precision.

>_ steamm/sources/bank/bank.move RUST

```
fun funds_deployed_<P, T, BToken>(
    bank: &Bank<P, T, BToken>,
    lending_market: &LendingMarket<P>,
): Decimal {
    // FundsDeployed =  cTokens * Total Supply of Funds / cToken Supply
    if (bank.lending.is_some()) {
        [...]
        decimal::from(bank.lending.borrow().ctokens).mul(ctoken_ratio)
    } else {
        decimal::from(0)
    }
}
```

4. Instead of splitting `fee_a` and `fee_b` in `fee::withdraw`, refactor it to utilize `withdraw_all`.
5. Implement `wrapping_add` functionality for the `lifetime_*` tracking values in `pool` to ensure that the values will wrap around rather than panic on overflow.

Remediation

Update the codebase with the above refactors.

Patch

1. Issue #3 resolved in 96f13c5.

Code Maturity OS-SAM-SUG-04

Description

1. Utilize the actual available balance values instead of the fee amounts from `quote.fees` in `pool::redeem_liquidity` if the available balance of `coin_a` (`balance_a_value`) and `coin_b` (`balance_b_value`) is less than `quote.fees`, to avoid a mismatch between actual deducted fees and reported values. This ensures that the `redemption_fees_a` and `redemption_fees_b` fields always reflect the actual fees collected.

>_ contracts/steamm/sources/pool/pool.move RUST

```
public fun redeem_liquidity<A, B, Quoter: store, LpType: drop>(
    pool: &mut Pool<A, B, Quoter, LpType>,
    lp_tokens: Coin<LpType>,
    min_a: u64,
    min_b: u64,
    ctx: &mut TxContext,
): (Coin<A>, Coin<B>, RedeemResult) {
    [...]
    // Update redemption fee data
    pool.trading_data.redemption_fees_a = pool.trading_data.redemption_fees_a +
        ↪  quote.fees_a();
    pool.trading_data.redemption_fees_b = pool.trading_data.redemption_fees_b +
        ↪  quote.fees_b();
    [...]
}
```

2. Utilize `checked_mul_div_up` in `cpmm::max_amount_in_on_a2b` to ensure the calculation always rounds up, favoring the pool and liquidity providers rather than the users.
3. In `bank`, `get_type_reflection` and `assert_btoken_type` rely on string-based type name reflection to validate type relationships. However, this method is not a reliable approach for type verification and should be utilized cautiously. That said, its current implementation does not pose a direct security risk.

>_ sources/bank/bank.move RUST

```
public(package) fun assert_btoken_type<T, BToken>() {
    let type_reflection_t = get_type_reflection<T>();
    let type_reflection_btoken = get_type_reflection<BToken>();

    let mut expected_btoken_type = string::utf8(b"B_");
    string::append(&mut expected_btoken_type, type_reflection_t);
    assert!(type_reflection_btoken == expected_btoken_type, EBTokenTypeInvalid);
}
```

4. To ensure that the multiplication in `bank_math::compute_amount_to_recall` and `bank_math::compute_amount_to_deploy` does not result in overflow and maintains higher precision, switch to utilizing `u128` for all calculations involving multiplication.
5. In `quoter_math::newton_raphson`, the comment for `tol` states the value is `1e-10`, but the actual value is `1e-14`. The value should either be adjusted or the comment updated for accuracy.
6. The `is_expo_negative` flag in `switchboard::from_switchboard_decimal` should be set to true to correctly interpret Switchboard prices as fixed-point decimals.

Remediation

Implement the above-mentioned suggestions.

Patch

1. Issue #1 resolved in 2b99003.
2. Issue #2 resolved in 5325116.
3. Issue #4 resolved in 71c2aed.

Code Clarity

OS-SAM-SUG-05

Description

1. In `quote`, `amount_out_net`, `amount_out_net_of_protocol_fees`, `output_fee_rate`, and `amount_out_net_of_pool_fees` perform subtraction on `amount_out` by the respective fees (`protocol_fees` and `pool_fees`). However, there is no validation to ensure that the amount of fees does not exceed the `amount_out`. If this occurs it will result in a underflow.
2. `quote::output_fee_rate` divides `total_fees` by `amount_out` (the amount of tokens to be swapped). However, there is no check to prevent division by zero if `amount_out` is zero.

>_ steamm/sources/quote.move RUST

```
public fun output_fee_rate(swap_quote: &SwapQuote): Decimal {
    let total_fees = decimal::from(
        swap_quote.output_fees().pool_fees() + swap_quote.output_fees().protocol_fees(),
    );
    total_fees.div(decimal::from(swap_quote.amount_out()))
}
```

3. In `bank`, the `EUtilisationRangeBelowHundredPercent` error seems to indicate an issue where the utilization rate is below 100%, however, the actual error is about indicating when the utilization range is below zero. Utilize the correct error.
4. The naming of `math::safe_compare_mul_u64` seems inaccurate, as according to the returned data, it is a greater than or equal to function, and not a compare function.

>_ steamm/sources/math.move RUST

```
public(package) fun safe_compare_mul_u64(a1: u64, b1: u64, a2: u64, b2: u64): bool {
    let left = (a1 as u128) * (b1 as u128);
    let right = (a2 as u128) * (b2 as u128);
    left >= right
}
```

Remediation

1. Add validation to ensure `amount_out` and `output_fees` calculations do not abort.
2. Assert `amount_out` is not zero in `quote::output_fee_rate`.
3. Utilize the correct error in `bank`.
4. Modify the name of `math::safe_compare_mul_u64` to accurately represent its functionality.

Patch

1. Issue #2 resolved in 0aa893f.
2. Issue #3 resolved in cc20935.
3. Issue #4 resolved in 1a36800.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.